

LAB NO: 07

LAB TITLE : One Dimensional Array in C++ .

OBJECTIVE :

The objective of this lab is to understand the concept of one-dimensional arrays in C++. This lab helps students learn how to declare arrays, initialize arrays, access array elements, copy one array into another array, and perform operations like finding the sum of elements using loops. Arrays are very useful in programming because they allow storing multiple values of the same data type using a single variable name.

THEORY :

An array is a collection of elements of the same data type stored in contiguous memory locations. Each element in an array is identified by an index number. In C++, array indexing starts from 0. Arrays are useful when we need to store large amounts of similar data.

For example

```
int marks[5];
```

Array Declaration :

To declare an array in C++, the programmer specifies the type of the elements and the number of elements required by an array as follows:

Syntax :

```
dataType arrayName[arraySize];
```

Array Initialization :

It is like a variable, an array can be initialized. To initialize an array, we provide initializing values which are enclosed within curly braces in the declaration and placed following an equals

sign after the array name. Here is an example of initializing an integer array

Syntax:

```
int age[4] = {10,20,30,40};
```

Accessing Array Elements :

In any point of a program in which an array is visible, we can access the value of any of its elements individually as if it was a normal variable, thus being able to both read and modify its value.

Syntax :

```
arrayName[index];
```

Copying Arrays :

In C++, arrays cannot be copied directly using one statement. We use loops to copy elements one by one from one array to another array.

SOFTWARE :

1. DEV-C++ IDE
2. C++ Compiler

LAB TASKS :

Array Declaration and Initialization

1. Declare an integer array called "numbers" with a size of 5.
2. Initialize the array with values 10, 20, 30, 40, and 50.
3. Display the values of the array elements using a loop.

```
#include <iostream>
using namespace std;

int main()
{
    int numbers[5] = {10, 20, 30, 40, 50}; // array ko initialize karna
```

```

    for (int i = 0; i < 5; i++)
    {
        cout << numbers[i] << endl; // array ke elements print karna
    }

    return 0;
}

```

Accessing Array Elements

1. Declare a character array called "message" to store a word of your choice.
2. Use a loop to access each character of the array and display it on a separate line.

```

#include <iostream>
using namespace std;

int main()
{
    char message[] = "HELLO"; // character array banana

    for (int i = 0; message[i] != '\0'; i++)
    {
        cout << message[i] << endl; // har character ko print karna
    }

    return 0;
}

```

Copying Arrays

1. Declare two integer arrays: "source" and "destination", each with a size of 5
2. Initialize the "source" array with some values.
3. Use a loop to copy the elements of the "source" array into the "destination" array.
4. Display the values of both arrays to verify the copy operation.

```

#include <iostream>
using namespace std;

int main()
{
    int source[5] = {1, 2, 3, 4, 5}; // source array banana
    int destination[5]; // destination array banana

    for (int i = 0; i < 5; i++)
    {
        destination[i] = source[i]; // array copy karna
    }

    cout << "Source Array:" << endl;

    for (int i = 0; i < 5; i++)
    {
        cout << source[i] << " "; // source array print karna
    }

    cout << endl;
    cout << "Destination Array:" << endl;

    for (int i = 0; i < 5; i++)
    {
        cout << destination[i] << " "; // destination array print karna
    }

    return 0;
}

```

Array Sum

1. Write a C++ program that prompts the user to enter 5 numbers.
2. Store the numbers in an integer array.

3. Calculate the sum of all the numbers in the array.

4. Display the calculated sum.

```
#include <iostream>
using namespace std;

int main()
{
    int numbers[5];
    int sum = 0;

    for (int i = 0; i < 5; i++)
    {
        cout << "Enter 5 numbers: ";
        cin >> numbers[i]; // user se values lena
    }

    for (int i = 0; i < 5; i++)
    {
        sum = sum + numbers[i]; // sum calculate karna
    }

    cout << "Sum = " << sum; // final sum print karna

    return 0;
}
```

CONCLUSION :

In this lab, we learned the basic concepts of one-dimensional arrays in C++. We studied array declaration, initialization, accessing elements, copying arrays, and calculating sum using loops. This lab improved our understanding of arrays and their practical implementation in programming.